Version: 1.0



Selection

C-Structures

Résumé

Implémentez des structures C fondamentales et des techniques de gestion de mémoire pour manipuler et stocker efficacement des données.

#C #Structures #MemMgmt



Avertissement relatif à la propriété intellectuelle

L'ensemble du contenu présenté dans ce module de formation — y compris, sans s'y limiter, les textes, images, graphiques et autres éléments — est protégé par les droits de propriété intellectuelle détenus par l'Association 42.

Conditions d'utilisation

- **Usage personnel :** L'utilisation du contenu de ce module est strictement réservée à un usage personnel. Toute reproduction, diffusion, modification, utilisation publique ou commerciale est interdite sans l'autorisation écrite préalable de l'Association 42.
- **Respect de l'intégrité :** Il est interdit de modifier, altérer ou adapter le contenu de manière à en dénaturer le sens ou à porter atteinte à son intégrité.

Protection des droits

Toute violation des présentes conditions constitue une atteinte aux droits de propriété intellectuelle et peut faire l'objet de poursuites. L'Association 42 se réserve le droit d'engager toute action nécessaire à la protection de ses droits, notamment par voie judiciaire et en réclamant des dommages et intérêts le cas échéant.

Pour toute question relative à l'utilisation du contenu ou pour obtenir une autorisation, veuillez contacter : `legal@42.fr`

Table des matières

1 Instructions 1

2 Préambule 5

3 Tutoriel : Tableaux 6

4 Tutoriel : Structures 8

5 Exercice 0 : ft_point.h 10

6 Exercice 1 : ft_strs_to_tab 11

7 Exercice 2 : ft_show_tab 13

8 Soumission et évaluation par les pairs 14

Chapitre 1

Instructions

- Seule cette page servira de référence : ne faites pas confiance aux rumeurs.
- Attention ! Ce document pourrait potentiellement changer jusqu'à la soumission.
- Assurez-vous d'avoir les permissions appropriées sur vos fichiers et répertoires.
- Vous devez suivre les procédures de soumission pour tous vos exercices.
- Vos exercices seront vérifiés et notés par vos camarades de classe.
- De plus, vos exercices seront vérifiés et notés par un programme appelé Moulinette.
- Moulinette est très méticuleuse et stricte dans son évaluation de votre travail. Elle est entièrement automatisée, et il n'y a aucun moyen de négocier avec elle. Donc, pour éviter les mauvaises surprises, soyez aussi minutieux que possible.
- Moulinette n'est pas très ouverte d'esprit. Elle n'essayera pas de comprendre votre code s'il ne respecte pas la Norme. Moulinette s'appuie sur un programme appelé `norminette` pour vérifier si vos fichiers respectent la norme. En bref : il serait stupide de soumettre un travail qui ne passe pas la vérification de `norminette`.
- Ces exercices sont soigneusement disposés par ordre de difficulté - du plus facile au plus difficile. Nous `ne considérerons pas` un exercice plus difficile réussi si un exercice plus facile n'est pas parfaitement fonctionnel.
- Utiliser une fonction interdite est considéré comme de la tricherie. Les tricheurs obtiennent -42, et cette note est non négociable.
- Vous n'aurez à soumettre une fonction main() que si nous demandons un programme.
- Moulinette compile avec ces flags : -Wall -Wextra -Werror, et utilise `cc`.
- Si votre programme ne compile pas, vous obtiendrez `0`.
- Vous ne pouvez pas laisser aucun fichier supplémentaire dans votre répertoire autre que ceux spécifiés dans le sujet.
- Vous avez une question ? Demandez à votre pair à droite. Sinon, essayez votre pair à gauche.
- Votre guide de référence s'appelle `Google / man / Internet / ...`.
- Consultez le Slack Piscine.
- Examinez attentivement les exemples. Ils pourraient très bien appeler à des détails qui ne sont pas explicitement mentionnés dans le sujet...

© 2026 Association 42. - Tous droits réservés

- Par Odin, par Thor ! Utilisez votre cerveau ! ! !



N'oubliez pas d'ajouter l'*en-tête standard 42* dans chacun de vos fichiers .c/.h. Norminette vérifie son existence de toute façon !



Norminette doit être lancée avec le flag *-R CheckForbiddenSourceHeader*. Moulinette l'utilisera aussi.

© 2026 Association 42. - Tous droits réservés

Contexte

La Piscine C est intense. C'est votre premier grand défi à 42 — une plongée profonde dans la résolution de problèmes, l'autonomie et la communauté.

Durant cette phase, votre objectif principal est de construire vos fondations — à travers la lutte, la répétition, et surtout l'échange d'**apprentissage par les pairs**.

À l'ère de l'IA, les raccourcis sont faciles à trouver. Cependant, il est important de considérer si votre utilisation de l'IA vous aide vraiment à grandir — ou si elle se contente de vous empêcher de développer de vraies compétences.

La Piscine est aussi une expérience humaine — et pour l'instant, rien ne peut remplacer cela. Pas même l'IA.

Pour un aperçu plus complet de notre position sur l'IA — en tant qu'outil d'apprentissage, dans le cadre du programme ICT, et en tant qu'attente croissante sur le marché du travail — veuillez vous référer à la FAQ dédiée disponible sur l'intranet.

Message principal

- Construire des fondations solides sans raccourcis.
- Développer réellement des compétences techniques et personnelles.
- Expérimenter un véritable apprentissage par les pairs, commencer à apprendre à apprendre et à résoudre de nouveaux problèmes.
- Le parcours d'apprentissage est plus important que le résultat.
- Apprendre les risques associés à l'IA, et développer des pratiques de contrôle efficaces et des contre-mesures pour éviter les pièges courants.

Règles pour les apprenants :

- Vous devez appliquer le raisonnement à vos tâches assignées, surtout avant de vous tourner vers l'IA.
- Vous ne devez pas demander de réponses directes à l'IA.
- Vous devez apprendre l'approche globale de 42 sur l'IA.

© 2026 Association 42. - Tous droits réservés

Résultats de la phase :

Dans cette phase de fondation, vous obtiendrez les résultats suivants :

- Acquérir des bases techniques et de codage appropriées.
- Savoir pourquoi et comment l'IA peut être dangereuse pendant cette phase.

Commentaires et exemple :

- Oui, nous savons que l'IA existe — et oui, elle peut résoudre vos activités. Mais vous êtes ici pour apprendre, pas pour prouver que l'IA a appris. Ne gaspillez pas votre temps (ou le nôtre) juste pour démontrer que l'IA peut résoudre le problème donné.
- Apprendre à 42 ne consiste pas à connaître la réponse — il s'agit de développer la capacité à en trouver une. L'IA vous donne la réponse directement, mais cela vous empêche de construire votre propre raisonnement. Et le raisonnement prend du temps, des efforts, et implique des échecs. Le chemin vers le succès n'est pas censé être facile.
- Gardez à l'esprit que lors des examens, l'IA n'est pas disponible — pas d'internet, pas de smartphones, etc. Vous réaliserez rapidement si vous avez trop compté sur l'IA dans votre processus d'apprentissage.
- L'apprentissage par les pairs vous expose à différentes idées et approches, améliorant vos compétences interpersonnelles et votre capacité à penser de manière divergente. C'est bien plus précieux que de simplement discuter avec un bot. Alors ne soyez pas timide — parlez, posez des questions, et apprenez ensemble !
- Oui, l'IA fera partie du programme — à la fois comme outil d'apprentissage et comme sujet en soi. Vous aurez même l'occasion de construire votre propre logiciel d'IA. Afin d'en apprendre davantage sur notre approche crescendo, vous passerez par la documentation disponible sur l'intranet.

✓ Bonne pratique :

> Je suis bloqué sur un nouveau concept. Je demande à quelqu'un à proximité comment il l'a abordé. Nous parlons pendant 10 minutes — et soudain, ça fait clic. Je comprends.

✗ Mauvaise pratique :

> J'utilise secrètement l'IA, je copie du code qui semble correct. Pendant l'évaluation par les pairs, je ne peux rien expliquer. J'échoue. Pendant l'examen — sans IA — je suis à nouveau bloqué. J'échoue.

© 2026 Association 42. - Tous droits réservés

Chapitre 2

Préambule

Voici ce que dit Wikipédia à propos de l'ornithorynque :

L'ornithorynque (Ornithorhynchus anatinus), aussi connu sous le nom de "platypus" ou "ornithorynque à bec de canard", est un mammifère semi-aquatique pondeur originaire de l'est de l'Australie, y compris la Tasmanie. Avec les quatre espèces d'échidnés, c'est l'un des cinq monotrèmes existants, les seuls mammifères qui pondent des œufs au lieu d'accoucher.

L'animal est le seul représentant vivant de sa famille (Ornithorhynchidae) et de son genre (Ornithorhynchus), bien que plusieurs espèces apparentées aient été retrouvées dans les fossiles.

L'apparence inhabituelle de ce mammifère pondeur à bec de canard, queue de castor et pattes de loutre a longtemps déconcerté les naturalistes européens, certains pensant qu'il s'agissait d'un canular. C'est aussi l'un des rares mammifères venimeux : le mâle possède un éperon sur la patte arrière capable d'infliger une douleur intense. Ses caractéristiques uniques en font un sujet d'étude important pour la biologie évolutive, et un symbole emblématique de l'Australie ; il apparaît souvent comme mascotte lors d'événements nationaux et figure sur la pièce de 20 centimes australienne. C'est l'emblème animalier de l'état de Nouvelle-Galles du Sud.

Jusqu'au début du XXe siècle, il fut chassé pour sa fourrure, mais il est désormais protégé sur tout son territoire. Bien que les tentatives de reproduction en captivité aient été peu concluantes, et que la pollution puisse l'affecter, l'espèce n'est pas immédiatement menacée.

Ce sujet ne parle absolument pas d'ornithorynques.

© 2026 Association 42. - Tous droits réservés

Chapitre 3

Tutoriel : Tableaux

- Jusqu'à présent, vous avez utilisé des types simples en `C` : `char`, `int`, etc. Commençons à utiliser des types plus complexes ! Nous allons commencer par les **tableaux**, puis les **structures**.
- Un **tableau** est une série d'objets de même type, tous situés de manière contiguë dans la mémoire de l'ordinateur : un `tableau` de `int`, un `tableau` de `char`, etc.
- Un tableau peut être créé directement en utilisant la syntaxe suivante :

main.c

```
int main(void)
{
    int my_int_tab[42];
}
```

Cela crée un tableau de 42 `int` en mémoire. Le contenu de chaque `int` est inconnu, comme pour toute création de variable.

- Vous pouvez accéder à chaque élément du tableau individuellement :

main.c

```
int main(void)
{
    int my_int_tab[42];

    my_int_tab[0] = 5; // Accès au premier élément
    my_int_tab[41] = 7; // Accès au dernier élément
}
```

- Vous pouvez également les manipuler comme une variable :

© 2026 Association 42. - Tous droits réservés

main.c

```
int main(void)
{
    int my_int_tab[42];
    int a;

    a = my_int_tab[21] + 1;
}
```

- Essayons ! Créez un fichier `C` avec un **tableau** de 5 `entiers`. Remplissez-le avec les valeurs 0, 1, 2, 3, 4 en utilisant une boucle. Affichez tous les éléments pour vérifier.
- Affichez le même tableau du dernier élément au premier. Utilisez une autre boucle allant à l'envers.
- Vous avez déjà rencontré des tableaux : lorsque vous définissez une chaîne de caractères en utilisant des guillemets doubles. « Hello » crée un **tableau** de 6 `char` (y compris le 0 final), et remplit automatiquement chaque caractère avec le code ASCII de chaque lettre de la chaîne.
- Les tableaux et les pointeurs sont étroitement liés. Créez un **tableau** de 3 `entiers longs` et un `pointeur` vers `long`. Remplissez le tableau en utilisant uniquement le pointeur (pas directement le nom du tableau). Affichez les résultats.



Vous ne savez pas par où commencer ici ? Vérifiez votre compréhension avec votre voisin, et lancez-vous !

- Une fois que vous avez votre `tableau`, écrivez deux fonctions : une pour remplir un **tableau** de `entiers`, une autre pour l'afficher. Les deux doivent recevoir le `tableau` en paramètre. Testez depuis `main`.

© 2026 Association 42. - Tous droits réservés

Chapitre 4

Tutoriel : Structures

- Un tableau est bien, mais il a un inconvénient : au lieu de regrouper des éléments de même type, vous voulez parfois regrouper des éléments de types différents. Comme un `int`, puis un `char`, puis un `long`.
- Pour les types `C` standard, vous avez déjà un nom de type spécifique et vous pouvez directement créer une variable de ce type, mais il n'y a pas encore de nom de type pour notre propre usage (1 `int`, 1 `char`, 1 `long`). Donc, créer une structure en `C` se fera en 2 étapes : d'abord, vous devez définir le nouveau type. Ensuite, vous pouvez créer des éléments en utilisant ce nouveau type (autant que vous voulez, tout comme vous pouvez avoir plusieurs `int` ou `char` dans votre code).
- Utilisons donc les **structures** pour regrouper différents types ensemble. Définissez une **structure** en dehors de toute fonction :

Description

```
struct s_learner {
    int age;
    char grade;
    long id;
};
```

Créez une variable de ce type : `struct s_learner learner1;`. Remplissez chaque variable comme suit : `learner1.age = 20;`. Affichez toutes les valeurs des membres.



Définir une structure ne réserve pas de mémoire - cela crée simplement un nouveau nom de type.

- Créez 2 variables du même type de structure. Remplissez-les avec des valeurs différentes. Affichez les deux pour vérifier que chacune a des valeurs de membres indépendantes.



Lorsque vous créez plusieurs variables, pensez-vous qu'elles utilisent le même espace RAM pour chaque membre, ou que l'espace RAM sera réservé pour stocker chacune de vos structures ?

© 2026 Association 42. - Tous droits réservés

- Créez un tableau de 3 éléments de `struct`. Remplissez tous les membres de tous les éléments. Affichez chacun en montrant : `index du tableau, âge, note`, et `id`.
- Expérimentez avec les `pointeurs de structure`. Créez un pointeur `ptr` vers votre `struct`, assignez-lui une adresse de `struct`, puis accédez aux membres en utilisant la syntaxe `ptr->age`.
- N'hésitez pas à expérimenter ! Vous pouvez mettre des `tableaux` à l'intérieur des `structures`, des `structures` à l'intérieur des `structures`, et même des pointeurs vers le même type de `struct` à l'intérieur d'une `struct`.

© 2026 Association 42. - Tous droits réservés

Chapitre 5

Exercice 0 : ft_point.h

	Exercice: 0	
ft_point.h		
Répertoire: *ex0/*		
Fichiers à Rendre: `ft_point.h`		
Autorisé: `None`		

- Créez un fichier `ft_point.h` permettant de compiler le code suivant :

main.c

```c
#include "ft_point.h"

void set_point(t_point *point)
{
    point->x = 42;
    point->y = 21;
}

int main(void)
{
    t_point point;

    set_point(&point);
    return (0);
}
```

© 2026 Association 42. - Tous droits réservés

Chapitre 6

Exercice 1 : ft_strs_to_tab

	Exercice: 1	
ft_strs_to_tab		
Répertoire: *ex*1/		
Fichiers à Rendre: `ft_strs_to_tab.c`		
Autorisé: `malloc, free`		

- Créez une fonction qui prend en argument un tableau de chaînes de caractères et sa taille.

Prototype :

```
struct s_stock_str *ft_strs_to_tab(int ac, char **av);
```

- Elle transformera chaque élément de `av` en une structure.
- La structure sera définie dans le fichier `ft_stock_str.h` qui vous sera fourni, comme suit :

ft_stock_str.h

```
typedef struct   s_stock_str
{
    int  size;
    char *str;
    char *copy;
}                t_stock_str;
```

  - `size` représente la longueur de la chaîne ;
  - `str` représente la chaîne d'origine ;
  - `copy` est une copie de la chaîne ;

- L'ordre de `av` doit être conservé.

© 2026 Association 42. - Tous droits réservés

- Le tableau retourné devra être alloué dynamiquement et son dernier élément devra avoir `str` égal à 0 pour marquer la fin du tableau.
- La fonction devra retourner un pointeur NULL en cas d'erreur.

© 2026 Association 42. - Tous droits réservés

Chapitre 7

Exercice 2 : ft_show_tab

	Exercice: 2	
ft_show_tab		
Répertoire: *ex2/*		
Fichiers à Rendre: `ft_show_tab.c`		
Autorisé: `write`		

- Créez une fonction qui affiche le contenu du tableau retourné par la fonction précédente.
- Voici son prototype :

Prototype :

```
void ft_show_tab(struct s_stock_str *par);
```

- La structure sera la même que dans l'exercice précédent, définie dans `ft_stock_str.h`.
- Pour chaque élément, il faudra afficher :
- la chaîne originale suivie d'un '\n'
- la taille suivie d'un '\n'
- la copie de la chaîne (qui pourrait avoir été modifiée) suivie d'un '\n'
- Votre fonction sera testée à l'aide de notre propre `ft_strs_to_tab` (exercice précédent). Faites en sorte qu'elle fonctionne avec cela !

© 2026 Association 42. - Tous droits réservés

Chapitre 8

Soumission et évaluation par les pairs

Déposez votre devoir dans votre dépôt `Git` comme d'habitude. Seul le contenu de votre dépôt sera évalué lors de la soutenance. N'hésitez pas à revérifier les noms de vos fichiers pour vous assurer qu'ils sont corrects.



Vous devez rendre uniquement les fichiers demandés par l'énoncé de ce projet.

© 2026 Association 42. - Tous droits réservés